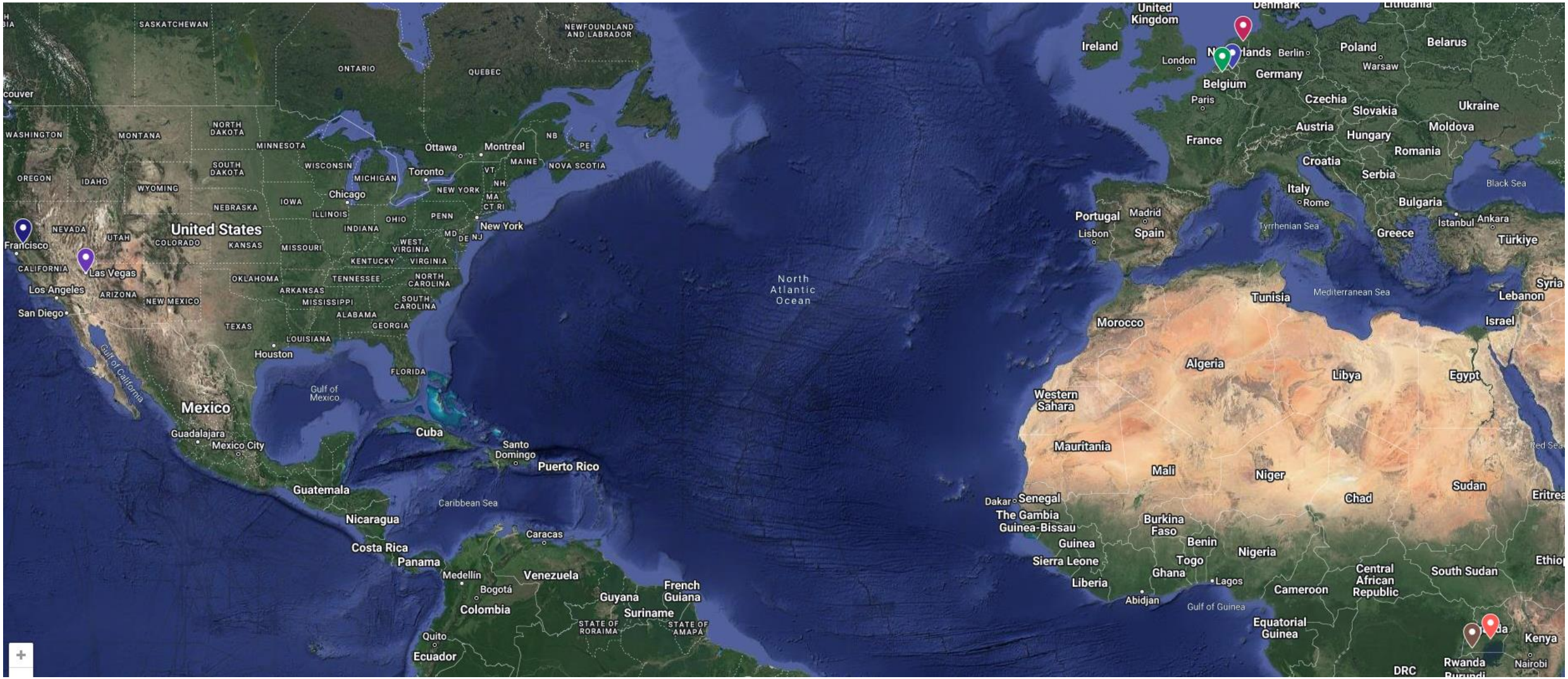
Software Product Design and Development I

Dr. John Businge
John.businge@unlv.edu

TA: Daniel Ogenrwot
ogenrwot@unlv.nevada.edu

My Journey to UNLV



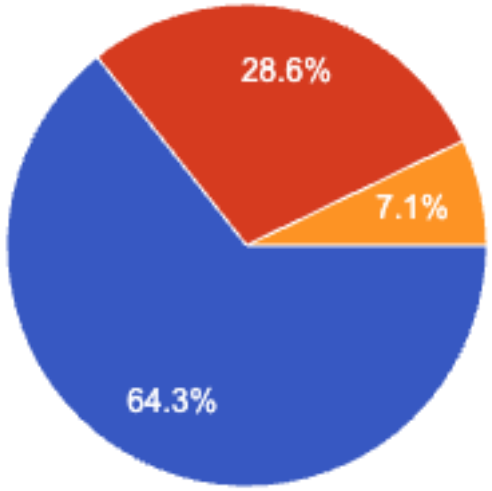
Administration

- Background Information survey.

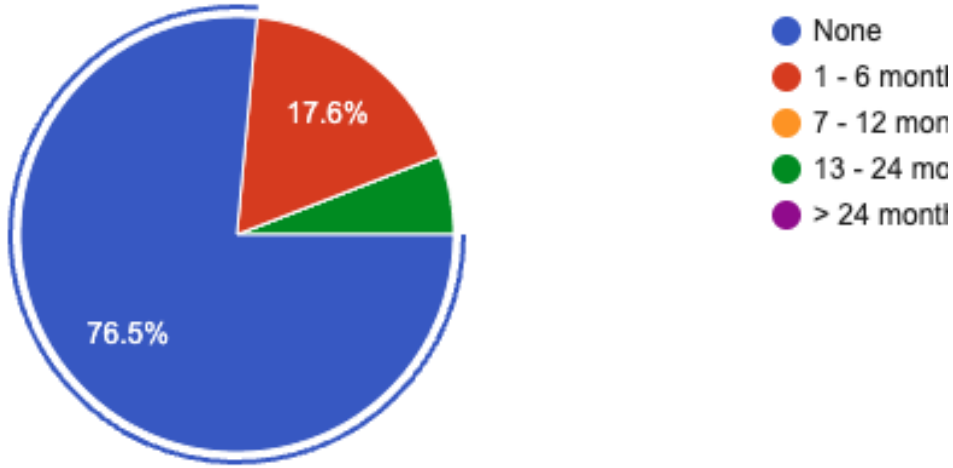
- Go to - https://johnxu21.github.io/teaching/CS472/
- My Lab - https://unlv-evol.github.io/

How many months of industry software development experience did you have before the beginning of class 472/672?

Fall 2025



Spring 2026



Course Goal: Build High-Quality Software Collaboratively

1. **Collaborate as a Team**
   Work effectively with teammates who bring different skills and responsibilities.

2. **Coordinate and Verify Contributions**
   Use GitHub, code reviews, testing, and CI to coordinate work and verify changes.

3. **Design for Shared Development**
   Design software that team members can develop, integrate, and maintain together.

Throughout the course, you will apply these practices to deliver a working MVP.

Course Learning Roadmap

Each stage prepares you to contribute more effectively to the team project.

1. **Git and GitHub**
   Coordinate work through issues, branches, pull requests, reviews, and merges.

2. **Software Testing and CI**
   Write tests and use automated checks to protect software quality.

3. **Collaborative Software Design**
   Use architecture and clear interfaces to support parallel development and integration.

4. **Team Project**
   Apply these practices together to develop, evaluate, and deliver a working MVP.

Instruction → Practice → Application in the team project

One Collaborative Engineering Loop

We will use the follwing workflow for each meaningful project change.

ISSUE
Define change

DESIGN
Plan impact

BUILD
Branch + code

TEST
Verify behavior

REVIEW
PR + CI

MERGE
Integrate

MERGE ↻ NEXT ISSUE

Repeat the loop for each meaningful project change.

Teams, Iterative Development, and Design Portfolios

PLAN THE MVP

Scope • requirements • architecture

BUILD AND TEST

≥3 substantive PRs
≥3 evidence-based reviews

INTEGRATE AND DELIVER

≥3 substantive PRs
≥3 evidence-based reviews

Individual contribution across DP II and DP III

Each member: ≥6 PRs • ≥6 reviews | Quality and relevance matter - not only the count.

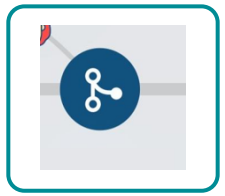
What Is a Minimum Viable Product?

The smallest usable version that addresses a core user need and can be meaningfully evaluated.

For this course, an MVP must satisfy three conditions:

VALUABLE

Addresses an important user need.



USABLE

Works as an integrated, end-to-end solution.

TESTABLE

Produces evidence and feedback for improvement.

Minimum scope—not minimum quality.

Build the MVP Through Usable Increments

Example: student assignment portal (a Canvas-like workflow)

COMPONENTS ONLY

Data store → Service API → Basic interface

Necessary components—but not yet a complete user outcome.

USABLE INCREMENTS

INCREMENT 1
View assignment
+ due date

INCREMENT 2
Submit work
+ confirmation

INCREMENT 3
Instructor review
+ feedback

Each increment works, provides value, and guides the next decision.

Choose Your Project Pathway

3 teams • 2 pathways • 1 informed decision

1

CLIENT-SPONSORED PROJECT

- Review the client's precondition report.
- Assess the scope, starting point, and constraints.
- Collaborate with the client throughout development.

2

TEAM-PROPOSED PROJECT

- Define a meaningful problem and intended users.
- Prepare the team's precondition report.
- Obtain instructor approval before development.

Choose based on the problem, feasibility, and team fit—not only the technology.

Why Student-Designed Projects Matter

They build ownership, product judgment, and technical leadership.

1

OWN THE PROBLEM

Choose a problem the team understands and genuinely wants to solve.

2

SHAPE THE PRODUCT

Define the users, requirements, priorities, and an achievable MVP.

3

LEAD THE ENGINEERING

Make, justify, and validate the important technical decisions as a team.

Success = a real need + a working product + evidence from users

Why Client Projects Matter

They add authentic needs, constraints, and accountability to the engineering work.

1

REAL NEEDS

Build for an actual organization and its users.

2

REAL CONSTRAINTS

Work within existing systems, access limits, and deadlines.

3

REAL COLLABORATION

Clarify expectations, communicate progress, and respond to feedback.

Success = a working product + a professional client relationship

Teamwork Challenges Become Project Risks

Recognize these patterns early—before they affect integration and delivery.

1

UNEVEN PARTICIPATION

A few members carry most of the work.

2

UNCLEAR OWNERSHIP

Tasks and dependencies are not clearly assigned.

3

LATE INTEGRATION

Changes collide near the deadline.

4

SILENT PROBLEMS

Risks remain hidden until recovery is difficult.

Make work, ownership, dependencies, and risks visible early.

Do Not Block the Team

Collaborative work depends on timely contributions and timely reviews.

1

BE READY EARLY

Open a complete, review-ready PR at least 24 hours before the deadline.

2

REVIEW PROMPTLY

Complete assigned reviews early enough for authors to respond.

3

REASSIGN—DON'T WAIT

Request another reviewer or review another available PR.

Different timelines, same responsibility

Labs: PR ready ≥24 hours early • Projects: ≥1 meaningful PR each week

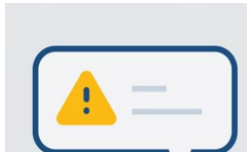
GitHub timestamps protect students who are ready on time.

Be ready, communicate, and keep the workflow moving.

Git and GitHub: A Shared Path from Task to Merge

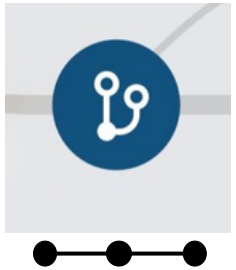
The workflow makes ownership, progress, and review visible.

ISSUE



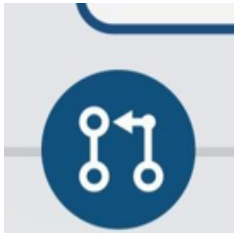
Agree on the task and ownership

BRANCH + COMMITS



Develop a focused change

PULL REQUEST



Explain the change and request review

REVIEW + MERGE

Improve it before it reaches main

The repository becomes the record of collaboration

Issues, commits, PRs, reviews, and merges show how the team worked.

Testing and CI: Three Questions Before We Merge

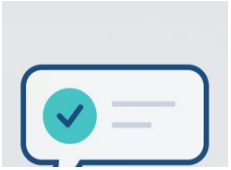
Each practice provides a different kind of evidence.

DOES IT WORK?

Write tests for expected behavior and important edge cases.

WHAT IS UNTESTED?

Use coverage to find important code the tests do not exercise.



WILL IT STAY CHECKED?

Use CI to run tests and quality checks on every pull request.

A green pipeline supports review

It provides fast feedback, but it does not replace thoughtful review.

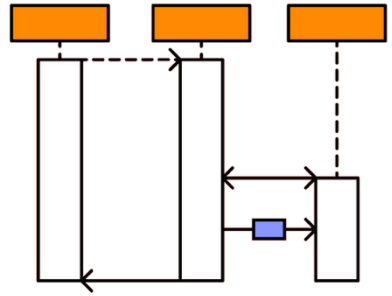
Collaborative Design: Decide What the Change Affects

Use only the design evidence needed to explain the contribution.

STRUCTURE

Are classes, responsibilities, or relationships changing?

Update the relevant class diagram.



BEHAVIOR

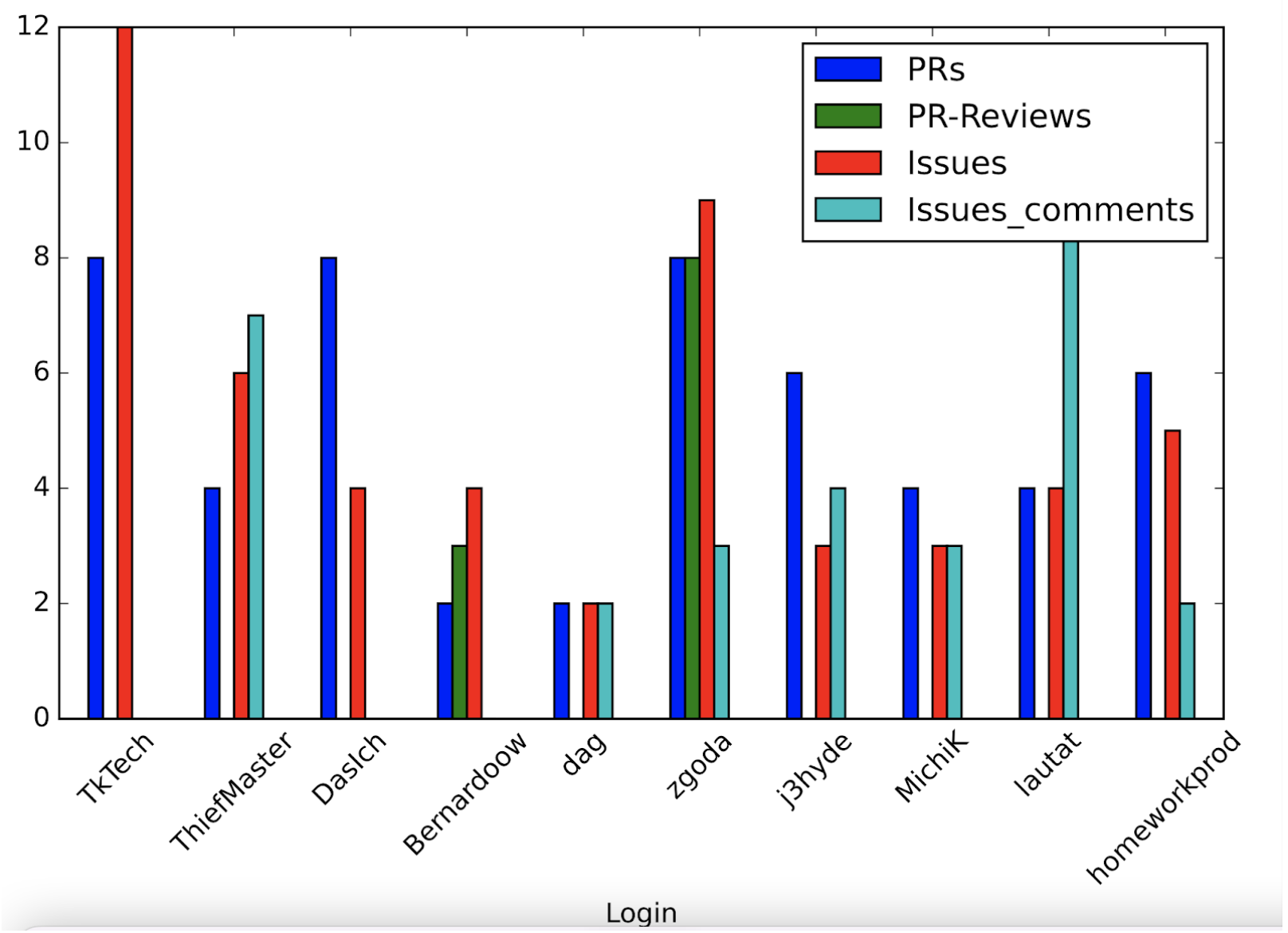
Are interactions between objects changing?

Update the relevant sequence diagram.

The PR connects design to implementation

Show the affected part of the system—not the entire system.

Assessment


12
10
8
6
4
2
0
PRs
PR-Reviews
Issues
Issues_comments
TKTech
ThiefMaster
DasIch
Bernardoow
dag
zgoda
j3hyde
MichiK
lautat
homeworkprod
Login

A list of previous projects by students

https://github.com/orgs/UNLV-CS472-672/repositories?type=all